

Assignment 4 Code Explanation

This document explains how the project matches the grading rubric for Assignment 4: MVC structure, MongoDB, Mongoose model, post display, post creation, asynchronous deletion with `fetch`, error handling, Git work, and README documentation.

Project Flow

The app starts in `server.js`. It loads environment variables from `.env`, creates the Express app, enables JSON/body parsing, configures sessions, serves static frontend files, mounts route files, connects to MongoDB through `connectDB()`, and then starts listening on the selected port.

Main Files and Responsibilities

File	Purpose	Important functions/lines
<code>server.js</code>	Application entry point.	Calls <code>connectDB()</code> ; mounts <code>pageRoutes</code> , <code>postRoutes</code> , <code>profileRoutes</code> , and <code>authRoutes</code> ; starts <code>app.listen()</code> .
<code>config/db.js</code>	MongoDB connection file.	<code>connectDB()</code> uses <code>mongoose.connect(process.env.MONGO_URI)</code> . On success it prints <code>MongoDB connected successfully</code> ; on failure it prints the error and exits.
<code>models/postModel.js</code>	Mongoose model layer.	Defines <code>postSchema</code> with required <code>title</code> , <code>content</code> , <code>author</code> , plus timestamps for <code>createdAt</code> and <code>updatedAt</code> .
<code>routes/postRoutes.js</code>	Post route layer.	Maps <code>GET /posts</code> , <code>POST /posts</code> , and <code>DELETE /posts/:id</code> to controller functions.
<code>controllers/postController.js</code>	Post controller layer.	<code>getPosts()</code> , <code>createPost()</code> , and <code>deletePost()</code> contain request logic and return JSON success/error responses.
<code>views/feed.html</code>	View layer for the feed.	Contains the post form, search input, feed message area, and <code>posts-list</code> container where posts are rendered.
<code>Website-for-ph-v4/scripts/Feed.js</code>	Client-side feed logic.	Uses <code>fetchPosts()</code> , <code>createPostFromForm()</code> , <code>displayPosts()</code> , <code>createPostCard()</code> , and <code>deletePost()</code> .

README.md	Run and structure documentation.	Explains setup, .env, MVC structure, post routes, delete flow, and Git rules.
-----------	----------------------------------	---

Function Locations

MongoDB Connection

config/db.js contains connectDB(). This is the only place responsible for connecting to MongoDB. It reads MONGO_URI from .env, which keeps the database connection string out of Git.

Post Model

models/postModel.js contains the Mongoose schema. The required assignment fields are:

- title - required string
- content - required string
- author - required string
- createdAt - created automatically by timestamps: true
- updatedAt - updated automatically by timestamps: true

Loading Posts

controllers/postController.js has getPosts(). It calls Post.find().sort({ createdAt: -1 }) and returns { success: true, posts }. This proves posts are loaded from MongoDB, not from an array in server memory.

Creating Posts

createPost() in controllers/postController.js validates title, content, and author. If one is missing, it returns status 400. If the data is valid, it calls Post.create() and returns the created post.

Deleting Posts

deletePost() in controllers/postController.js handles DELETE /posts/:id. It checks whether the id is a valid MongoDB ObjectId. Then it calls Post.findByIdAndDelete(req.params.id). If no post exists, it returns { success: false, message: "Post not found" } with status 404. If deletion succeeds, it returns { success: true, message: "Post deleted successfully" }.

Browser Fetch Delete

`Website-for-ph-v4/scripts/Feed.js` contains the browser-side `deletePost(postId)` function. It sends:

```
fetch(`/posts/${postId}`, { method: "DELETE" })
```

If the response is successful, it removes the post element from the screen with `document.getElementById(`post-${postId}`).remove()`. This satisfies the requirement that deletion happens without refreshing the page.

Rubric Mapping

Rubric item	Points	Where it is handled
Correct MVC structure	25	<code>models/</code> , <code>controllers/</code> , <code>routes/</code> , <code>views/</code> , <code>config/</code> , and cleaned <code>server.js</code> .
Correct MongoDB connection	15	<code>config/db.js</code> and <code>.env</code> with <code>MONGO_URI</code> . Verified by <code>MongoDB</code> connected successfully.
Correct Post model	10	<code>models/postModel.js</code> defines required fields and timestamps.
Display posts from database	10	<code>GET /posts</code> in <code>routes/postRoutes.js</code> , <code>getPosts()</code> in <code>postController.js</code> , and <code>fetchPosts()</code> in <code>Feed.js</code> .
Delete post from server and MongoDB	15	<code>DELETE /posts/:id</code> route and <code>Post.findByIdAndDelete()</code> in <code>deletePost()</code> .
Delete from screen using fetch	10	<code>deletePost(postId)</code> in <code>Feed.js</code> sends <code>fetch</code> and removes the DOM element.
Error handling	5	Controller returns 400 for missing required fields, 404 for missing posts, and 500 for server errors.
Correct Git work	5	Feature branch <code>feature/assignment-4-cleanup</code> , pushed branch, PR link, and <code>docs/git-strategy.pdf</code> .
README, order, documentation	5	<code>README.md</code> , <code>docs/git-strategy.pdf</code> , <code>docs/mongodb-verification.txt</code> , and this PDF.

Verification Evidence

The local MongoDB test succeeded with the following results, saved in `docs/mongodb-verification.txt`:

- `MongoDB` connected successfully
- Server is running on `http://localhost:3000`
- `GET /posts` success: `True`
- `POST /posts` success: `True`

- DELETE created post success: True
- Deleted post still exists after refresh-style GET: False
- DELETE fake post status: 404

What to Submit

- All project code files.
- README.md .
- docs/git-strategy.pdf .
- docs/assignment-4-code-explanation.pdf .
- Screenshot/output of MongoDB connection.
- Screenshots of branch, commits, PR, and partner approval.

Do not submit or commit .env .